Modular

TECHNICAL DEEP DIVE

# Journey of a Token From Cloud to Kernel

Overview of the Modular stack

Abdul Dakkak · Chief Scientist

# What is this presentation?

## A very high-level overview

The Modular stack end to end, and a teaser of the tech talks scheduled over the next few weeks.

SCHEDULE AND RECORDINGS

`go/ModularTechStack`

| TECH TALK | DATE |
|---|---|
| Overview of Modular stack | Sep 16, 2026 |
| Mojo compiler and stdlib | Sep 30, 2026 |
| Kernels | Oct 7, 2026 |
| Graph compiler, runtime and device context | Oct 14, 2026 |
| MAX framework | Oct 21, 2026 |
| MAX Serve | Oct 28, 2026 |
| MAX and Mojo tooling | Nov 4, 2026 |
| Distributed inference | Nov 18, 2026 |
| Performance engineering | Dec 2, 2026 |

# 01

# Cloud GenAI inference landscape

# GenAI changed how inference is deployed in the cloud

## 2012 – 2023

**APPLICATIONS**
Search · Ads · Social

**MODELS**
Small & open

**PLATFORM / INFRASTRUCTURE**
Managed ML — SageMaker, Vertex AI
Frameworks — TensorFlow, PyTorch, ONNX RT

**CLOUD**
Hyperscaler — AWS, GCP, Azure

**HARDWARE ACCELERATORS**
Intel · AMD · ARM · NVIDIA

## 2023 – TODAY

**APPLICATIONS**
Search / Chat · Coding · Social

**MODELS**
Larger & more complex · open + proprietary

**MODERN INFERENCE STACK**
Inference services — MCloud, TogetherAI, Fireworks
Frameworks — MAX, vLLM, SGLang

**CLOUD**
Hyperscaler · Neocloud · Sovereign

**HARDWARE ACCELERATORS**
NVIDIA · AMD · TPU · Trainium · Huawei

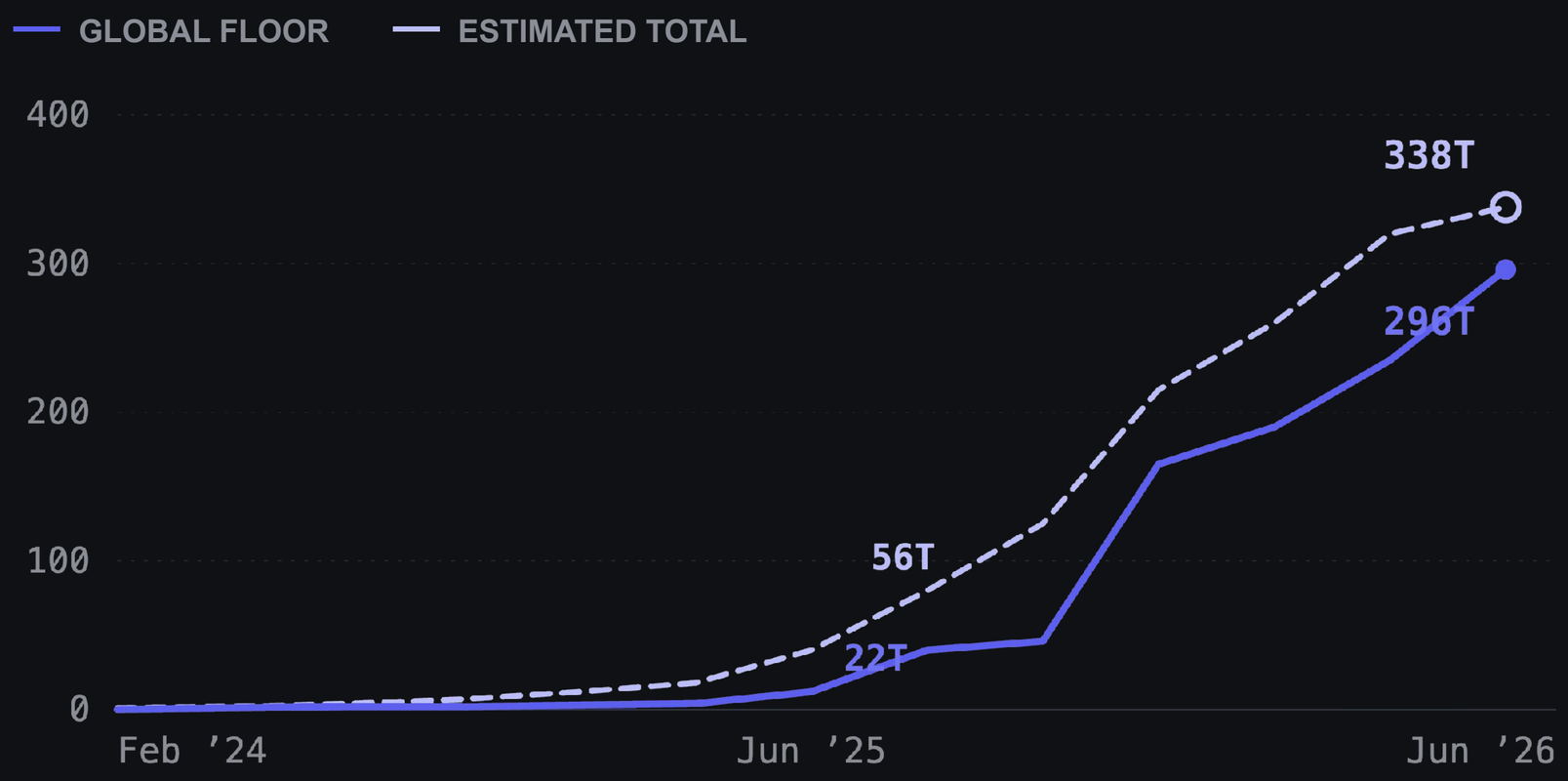
# Token consumption is growing exponentially

**6×**

growth of global token volume in a year

**~340T**

tokens / day globally, across all providers




Source: tokensperday.com

# The top 10 models account for 70%+ of total tokens

**70%**

of all routed tokens are served by 10 models with mostly text output

**95%**

of tokens are consumed by the top 50 models

**7 of 10**

top models are open-weight, six of them Chinese; the US closed frontier carries ~22% of top-10 tokens

Source: OpenRouter rankings

# 02

# Modular platform & business 101

# Turn GPU-hours into billable tokens

Margin improves by improving tokens per minute per Accelerator.

**GPU hours**

Input cost · 3rd-party cloud rentals

→

**Token factory**

Software platform — SLAs, utilization, reliability

→

**Billable tokens**

Revenue · delivered through the endpoint

Tokens / $ ↑

Optimizing tokens/$ can be done via performance optimizations or hardware optionality

# Modular’s answer: a multi-vendor alternative

Unify the software stack and commoditize the hardware.

| | |
|---|---|
| APPLICATIONS | Search / Chat · Coding · Social |
| MODELS | Larger & more complex · open + proprietary mix |
| **INFERENCE SERVICES** | Modular Cloud — cluster-level, proprietary |
| **FRAMEWORKS** | MAX — open & free<br>Mojo, the systems language — open & free |
| CLOUD PROVIDERS | AWS · GCP · OCI · Neoclouds |
| HW ALLIANCE · SHARED ECOSYSTEM | NVIDIA · AMD · TPU · Trainium · Qualcomm |

# The Modular stack: cloud, framework, language

| | |
|---|---|
| **Modular Cloud** | Managed inference — or bring your own cloud |
| **MAX** | Serving and modeling APIs · graph compiler · kernels |
| **Mojo** | The systems language for the AI era |
| **Hardware** | NVIDIA · AMD · Apple · Qualcomm · AWS · Google · Intel · Arm |



Modular Cloud, or BYOC

Serving & Modeling APIs

Mojo🔥

Mojo: The systems language for the AI era

NVIDIA AMD Q

aws G intel arm

03

# Why Mojo?

Today is no general purpose programming language the gives you performance and portability across hardware.

### THE PROBLEM

Kernels are written in CUDA, HIP, Triton, NKI or C++ (a language per vendor) and none of them the language familiar to AI researchers.

### THE DECISION

A Pythonic systems language built on MLIR with compile-time metaprogramming, library-defined primitives, and heterogeneity in mind.

### THE CONSEQUENCE

Every kernel in MAX is written once, in Mojo, and specialises to NVIDIA, AMD, Apple, Qualcomm and CPUs at compile time.

# Mojo, the systems language for the AI era, in five ideas

01

### A member of the Python family

Python syntax, extended only where performance demands it.

02

### Compile-time metaprogramming

Generic code that specialises to hardware, shape and dtype at no runtime cost.

03

### Abstractions in libraries, not the compiler

A library is far cheaper to change than a compiler.

04

### Built on MLIR

Built on the latest and greatest in compiler technology. This gives you performance, portability, and support.

05

### Heterogeneous compilation

One language, one program across all hardware.

# Mojo is syntactic sugar for MLIR

stdlib/std/builtin/simd.mojo

```
# a dialect op, wrapped in a normal constructor
self._mlir_value = __mlir_op .`pop.simd.splat`[
    _type=Self._mlir_type
](value._mlir_value)

# same mechanism, upstream dialects
__mlir_op .`index.cmp`[
    pred= __mlir_attr .`#index<...>`
](lhs_index, rhs_index)
```

### MLIR-native throughout

A dialect operation, type or attribute can be spelled inline in a .mojo file and wrapped in an ordinary function.

### Int is not a keyword

It is a struct in the standard library. The philosophy is that if a primitive can be written in Mojo, then it is not put in the compiler.

### Flexibility

Reaching a new instruction is a wrapper in a library file, not a compiler change. Community members brought up several consumer GPUs this way.

# The metaprogramming language is Mojo

```
map 64 threads onto a 16×64 output tile

# each thread's slice, resolved at compile time
comptime thread_layout = Layout.row_major(8, 4)
var  my_elements = output_tile.vectorize[1, 2]()
                    .distribute[thread_layout](lane_id())

comptime for i in  range(my_elements.shape[0]()):
  for j in  range(my_elements.shape[1]()):
        my_elements[i, j] = accumulator[i * stride + j]
```

### No separate template language

`comptime` marks work the compiler performs: calculations, conditionals, explicit unrolling, computed constraints.

### Lift compilation into the compiler time

What reaches the GPU is a handful of stores at constant offsets. The tiling, the vectorisation and the thread mapping will be folded away at compile time.

# Host code and device code in the same language

program.mojo — single source file

```
comptime Vec = TileTensor[dtype, type_of(layout)]

# device code
def  vector_add(left: Vec, right: Vec, out: Vec):
    var  idx = thread_idx.x
    out[idx] = left[idx] + right[idx]

# host code — same file, same language & semantics
ctx.enqueue_function[vector_add](
    left_tensor, right_tensor, out_tensor,
    grid_dim=1, block_dim=WIDTH,
)
ctx.synchronize()
```

# The Mojo internal dialects

```
!pop.int_literal                    # comptime integer — arbitrary precision
!pop.float_literal                  # comptime float — special forms + rationals
!kgen.string                        # comptime string
#kgen.param.decl.ref<"size">        # a parameter awaiting elaboration
!lit.struct<@stdlib::…::@UInt>      # a stdlib struct, as the front end sees it
```

### Library structs over primitives

Int, SIMD and pointers are library structs wrapping pop/kgen primitives.

### These dialects are ours

Mojo depends on no upstream MLIR dialect other than `llvm`. Reaching an upstream dialect is available but we do not depend on them.

### The language expands at the library level

A separation of concerns where the library engineers can work on the abstractions in the source language rather than in the compiler.

# What the accelerator finally sees

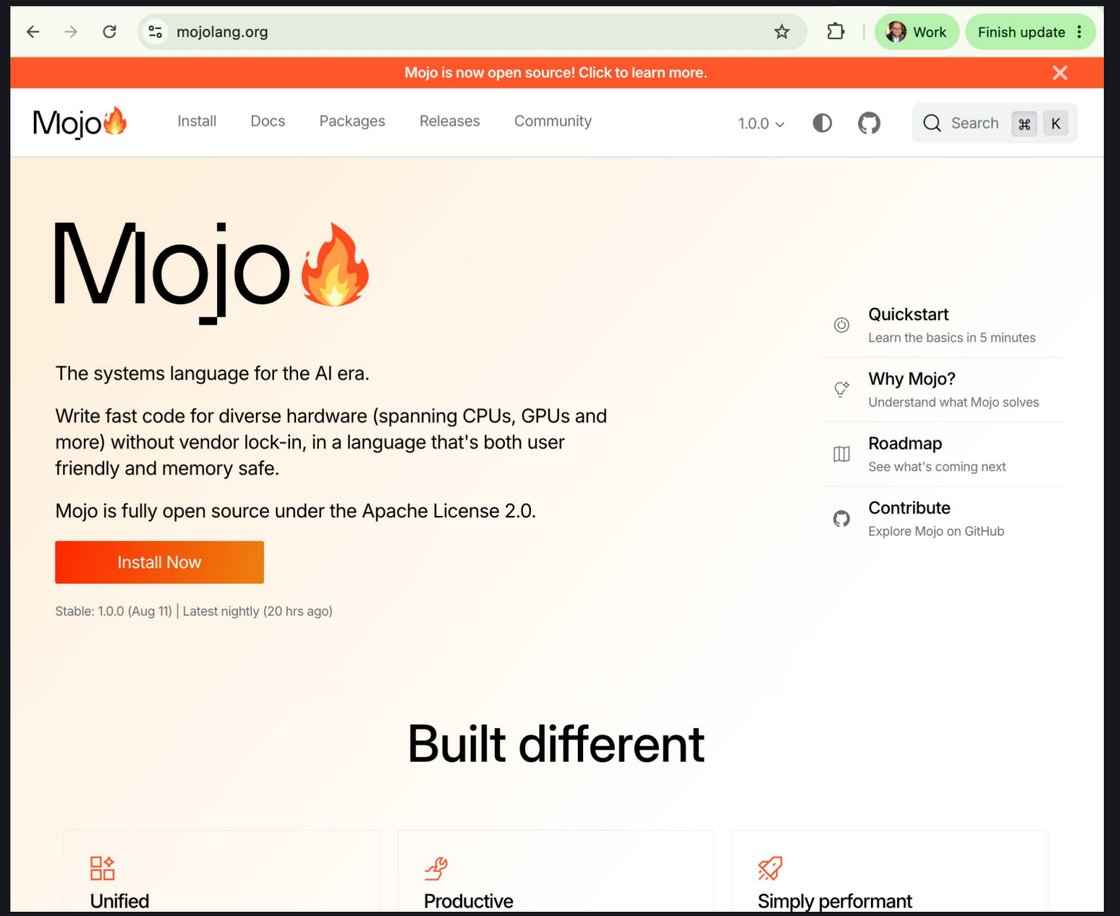
```
// PTX (illustrative) — the inner loop of a fused attention tile
cp.async.cg.shared.global   [smem_k], [gmem_k], 16;       // prefetch next tile
ldmatrix.sync.aligned.m8n8.x4.shared.b16  {r0,r1,r2,r3}, [smem_q];
mma.sync.aligned.m16n8k16.row.col.f32.bf16.bf16.f32 {d0,d1,d2,d3}, {a0,a1,a2,a3}, {b0,b1},
                                                    {c0,c1,c2,c3};
bar.sync 0;                                               // stage boundary
```

# Try Mojo today 🔥

```
$ pip install mojo
```

```
def main():
    print("Hello Mojo!")
```

mojolang.org — install, docs, packages, releases



# The Modular stack: cloud, framework, language

| | |
|---|---|
| **Modular Cloud** | Managed inference — or bring your own cloud |
| **MAX** | Serving and modeling APIs · graph compiler · kernels |
| **Mojo** | The systems language for the AI era |
| **Hardware** | NVIDIA · AMD · Apple · Qualcomm · AWS · Google · Intel · Arm |

# 04

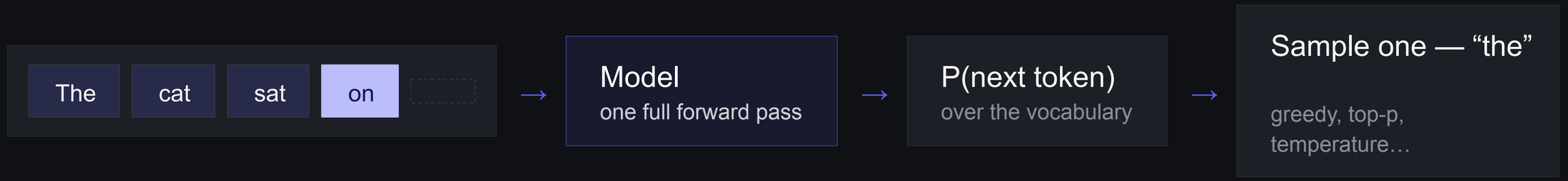
# A refresher on LLMs

# An LLM model is a sequence of ops



# LLMs run in an autoregressive manner

CONTEXT — GROWS BY ONE EACH PASS




↳ append into the empty slot — and run the whole thing again

repeat until EOS, a stop string, or the length cap

## One pass, one token

A 500-token answer is 500 full forward passes.

## Prompt vs. generation

Prompt tokens are known up front and process in parallel.

# Two phases of execution

## Prefill — read the prompt

All prompt tokens in one parallel pass. Large matrix multiplies, high arithmetic intensity: compute-bound. Produces the first token — and the KV cache. Sets time-to-first-token.

## Decode — generate the answer

One token per step. Each step re-reads every weight and the whole cache to do a little math: memory-bound. Sets inter-token latency, and runs hundreds of times per request.

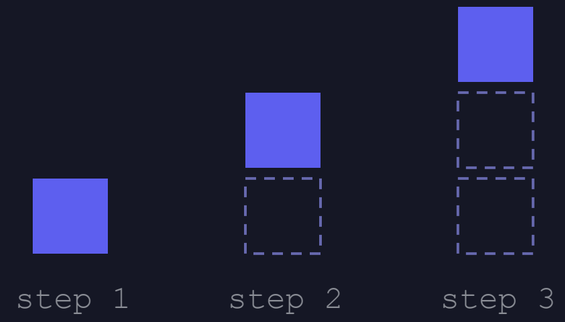
# The KV cache introduces state

### Without cache — recompute everything, every step




every step — K, V computed again; work grows with the square of the length

### With cache — compute once, read forever




dashed = read from cache · solid = the only K, V computed this step

Attention needs the K and V of every earlier token. Cache them, and each decode step computes exactly one new column — a few KB per token, GBs for long chat sessions.

# 05

# MAX kernels

# Why the MAX kernel library?

The primitives of modern inference, written once in Mojo without use of vendor binaries.

### THE PROBLEM

A precompiled vendor library gives you the operations and epilogues it shipped with. You cannot read inside it, and you cannot fuse across it.

### THE DECISION

One implementation per atomic operation — attention, matmul, norms, sampling, collectives — parameterised by target, shape and dtype.

### THE CONSEQUENCE

A new fusion, dtype or accelerator is a change in a library file, and it lands on every supported target at once.

# One library, all in Mojo, for every primitive operation

matmul | attention | paged attention | grouped MoE matmul | rms_norm

layer_norm | rope | fp8 · nvfp4 · mxfp4 gemm | top-k sampling

allreduce · allgather | scatter / gather | conv · pooling | softmax

elementwise fusion

… and the rest of modern LLM inference

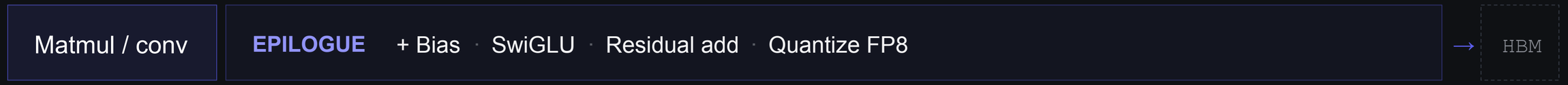
# MAX + Mojo give kernel authors super powers

UNFUSED — 5 LAUNCHES · 4 FULL ROUND-TRIPS THROUGH HBM




↓ compiler folds the suffix into the epilogue

SYNTHESIZED — 1 LAUNCH · ONE WRITE OUT, THE TAIL RUNS IN REGISTERS




**Why it needs a compiler.** Fusing into the epilogue means generating the matmul for this model. A precompiled vendor library only gives you the epilogues it shipped with.

# 06

# MAX modeling APIs and graph compiler

# Why the graph APIs and the compiler?

Define the model in Python then compile, optimize, and store the artifact.

### THE GRAPH-BUILDING APIS

PyTorch-like modules that build the same graph lazily, with symbolic shapes checked at build time. You iterate eagerly, then hand over one complete graph — `model.compile()`, not a rewrite.

### THE GRAPH COMPILER

Performs kernel synthesis, optimizations, plan buffers, and device placement. Has a global view of the model and decides the optimal schedule of execution.

# You define a model in Python via the MAX Graph APIs

```python
from max.experimental import nn from max.experimental
import functional as F

class  MLP(nn.Module):
   def  __init__(self, d, h):
        super().__init__()
        self.fc   = nn.Linear(d, h)
        self.proj = nn.Linear(h, d)

   def  forward(self, x):
        return self.proj(F.gelu(self.fc(x)))
```

**Eager API and graph construction**

Eager, PyTorch-like iteration builds the same graph lazily; model.compile() makes it ahead-of-time.

```python
model = MyModel(config)

# development: eager, inspectable
logits = model(tokens)

tokens_type = TensorType(DType.int64,
                         ["batch",  "seq" ],
                         device=Accelerator())

# production: same forward(), one graph
compiled = model.compile(tokens_type)
```

**Ahead-of-time compilation**

Shapes can be symbolic — “batch”, “seq” — checked algebraically at build time, guarded at run time.

# Thousands of models available

A library of architectures — Llama, Gemma, Qwen, DeepSeek, FLUX, hundreds more.

Architecture code + checkpoint = model: safetensors / GGUF weights load straight from Hugging Face.

Bring your own model and use it via `--custom-architectures` without forking the engine.

# What the IR looks like

```
mo.graph  @simple_add_graph<N>(%arg0: !mo.tensor<[N, 32], f32>,
                               %arg1: !mo.tensor<[N, 32], f32>) -> !mo.tensor<[N, 32], f32> {
  %0 =  rmo.add  (%arg0, %arg1)
       : (!mo.tensor<[N, 32], f32>, !mo.tensor<[N, 32], f32>) -> !mo.tensor<[N, 32], f32>
   mo.output  %0 : !mo.tensor<[N, 32], f32>
}
```

### This is MLIR

Shapes and dtypes live in the types; ops belong to named dialects.

### Two dialects already visible

`mo` is the container and type system; `rmo` the op you wrote.

# Five representations, from your model to an executable

| | | |
|---|---|---|
| RMO | **Relaxed MO** | The API-facing dialect. Rich ops, forgiving shapes. Each RMO op lowers to a composition of MO operators; shape constraints become symbolic runtime checks. |
| MO | **Modular Operators** | RISC-y — a deliberately small operation set with strict shape semantics. Terse, which is exactly what makes it optimizable. |
| MOGG | **Modular Generators** | Where the graph meets the kernel library. Ops bind to registered Mojo generators, parameterised by target and dtype. Fusion decisions land here. |
| MGP | **Modular Primitives** | Execution and flow. Buffers, allocation and reuse, streams, device placement, launch order, transfers. |
| MEF | **Modular Execution Format** | The compiled artifact the runtime loads and runs — the thing `max serve` actually holds. |

# MGP: from mathematics to execution

MGP — illustrative

```
%buf0 = mgp.alloc : memref<...> device(gpu:0)
mgp.launch  @fused_norm_qkv(%in, %w, %buf0)
    stream(%s0) device(gpu:0)
mgp.launch  @paged_flash_attention(%buf0, %kv)
    stream(%s0) device(gpu:0)
mgp.transfer  %kv_block -> host
    stream(%s1)
# the copy stream
mgp.dealloc %buf0   # reused downstream
```

### Memory planning across the whole model

Buffer lifetimes are known, so allocations are planned and reused rather than requested. That is what “whole-model compilation” means in practice.

### Streams and devices are explicit

Compute and copies live on separate streams — what lets prefix-cache loads overlap execution.

### Then MEF, and the driver

The MAX driver routes MEF commands to whichever backend is present: CUDA, ROCM, Metal, Qualcomm, Trainium, TPU.

INTERLUDE

# Interlude: the runtime

# Why the runtime exists

To maximize model performance, you need to ensure there is no overhead between operations.

### THE PROBLEM

A blocking call, a host callback or a cache load on the critical path stalls the device queue which decreases the accelerator utilization.

### THE DECISION

AsyncRT: every unit of work is a task with an async handle, dependents attach to the chain, and copies ride their own streams.

### THE CONSEQUENCE

All execution is async by default and Prefix-cache loads, sampling and KV transfer overlap execution instead of serializing.

# MEF is our executable format

```
compile once, initialise many times

artifact = model.compile(...)

artifact.export_mef( "model.mef")

# later, or elsewhere, or on another box
models = session.load_all("model.mef" )
models[ "vision_encoder" ]
models[ "language_model"]
```

### Compile and bind separately

Compilation and weight binding are separate steps, so one compiled artifact is reused across many initialisations.

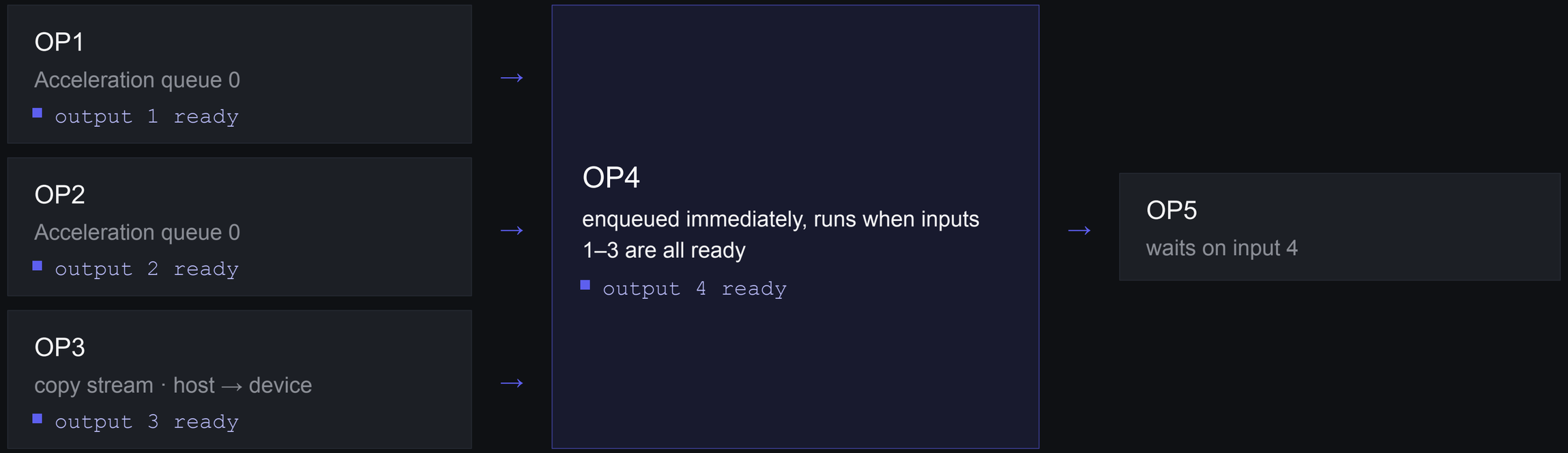
### One artifact, several models

A vision encoder and a language model compile together into one .mef, each keyed by its graph op — what a disaggregated encoder stage needs.

### No device required

Serialisation happens without initialising on hardware, so cross-compilation works. Build the artifact for a chip you do not have in the room.

# AsyncRT: everything is asynchronous by default




A task is enqueued with its inputs as futures, not called. It runs the moment every one of those inputs is ready; finishing resolves the future its dependents hold, and they run in turn — no polling, no blocking wait.

# 07

# MAX Serve

# What is MAX Serve?

An OpenAI-compatible endpoint with the serving optimisations inside the framework, not bolted on top of it.

### THE PROBLEM

Continuous batching, paged and prefix caching, chunked prefill and speculative decoding are what make a accelerator have high utilization. These optimizations usually live in a layer above the engine.

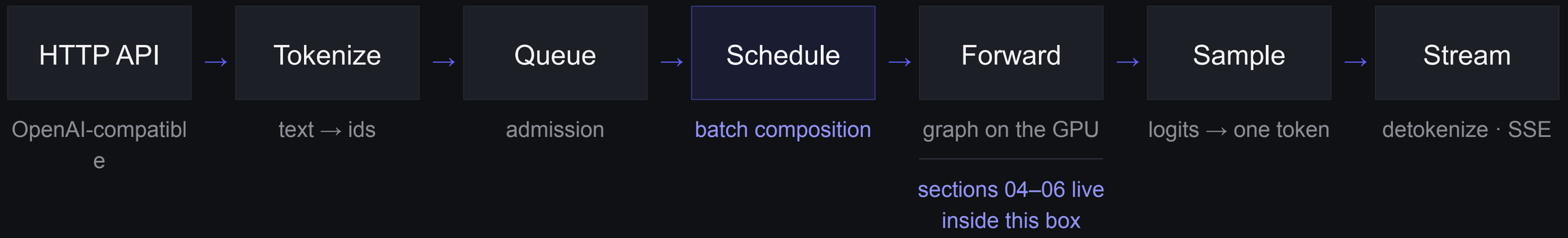
### THE COMPONENT

A local server that owns admission, scheduling, cache management, sampling and streaming, in the same framework as the compiler and the kernels.

### THE CONSEQUENCE

The scheduler can see what the compiler did, and it all ships as one container with no vendor toolkit.

# MAX Serve is a local server



# The scheduler runs to keep the accelerator busy

01 Admit what fits

02 Look up cached prefixes

03 Build one mixed batch

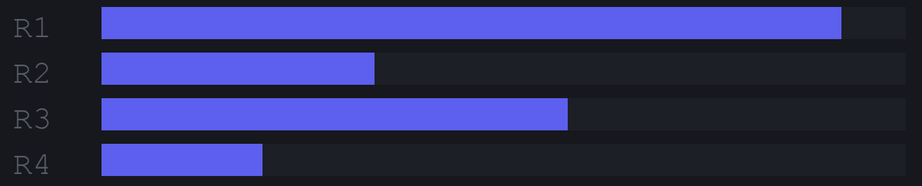
04 Run the graph once

05 Sample, stream, retire

WHY CONTINUOUS, NOT STATIC

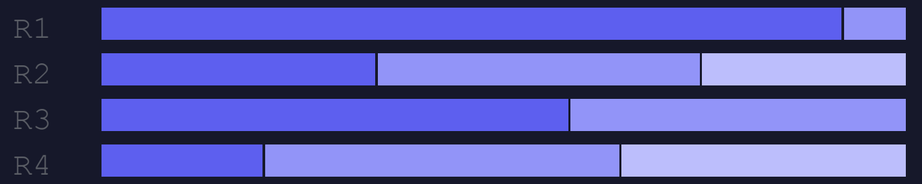
running slot idle

**Static batching** — everyone waits for the longest sequence




Three slots sit idle until the slowest request finishes — GPU-hours paid for and not billed.

**Continuous batching** — a free slot is refilled next step

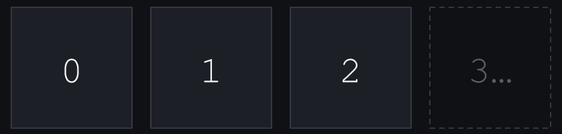



Lighter fills are requests admitted mid-flight: same four slots, no idle time, more tokens per GPU-hour.

# Paged KV-cache

Fixed-size pages let the cache grow one page at a time instead of reserving the maximum context length up front. As a result, nothing is stranded by fragmentation, and a shared prefix is one page referenced twice rather than two copies.

**SEQUENCE — LOGICAL**




grows page by page, allocated on demand

→

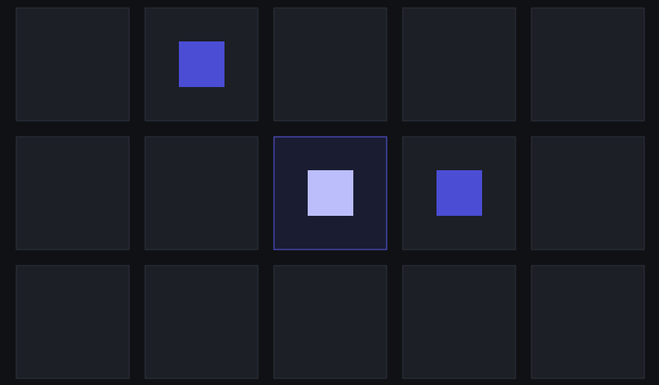
**BLOCK TABLE**

```
logical 0 → page 7
logical 1 → page 2
logical 2 → page 9
```

…per sequence, tiny

→

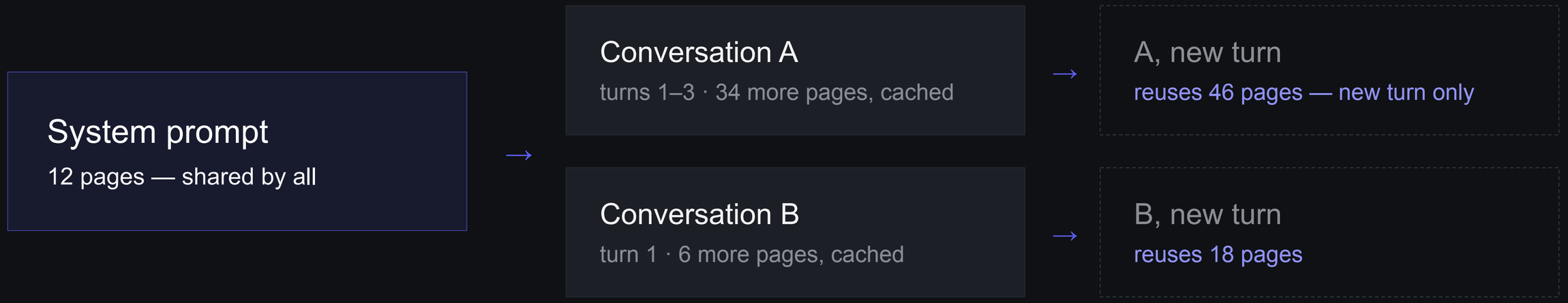
**PHYSICAL POOL — ACCELERATOR MEMORY**



highlighted page: referenced by two sequences — shared prefix, copy-on-write on divergence

# Prefix caching

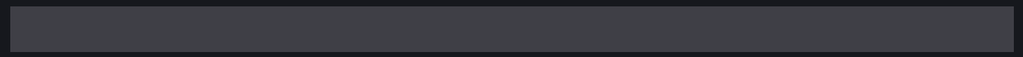
Requests share their opening — the system prompt, then every earlier turn of the conversation. Keep those pages and match them by hash, and a new turn prefills only what is actually new.



# Chunked prefill keeps one long prompt from stalling everyone

### One prefill owns the step

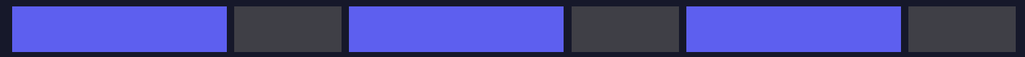
A single 200k-token prefill takes the whole step timeline — every decode in flight waits.



one step — prefill only

### Chunked, mixed into every step

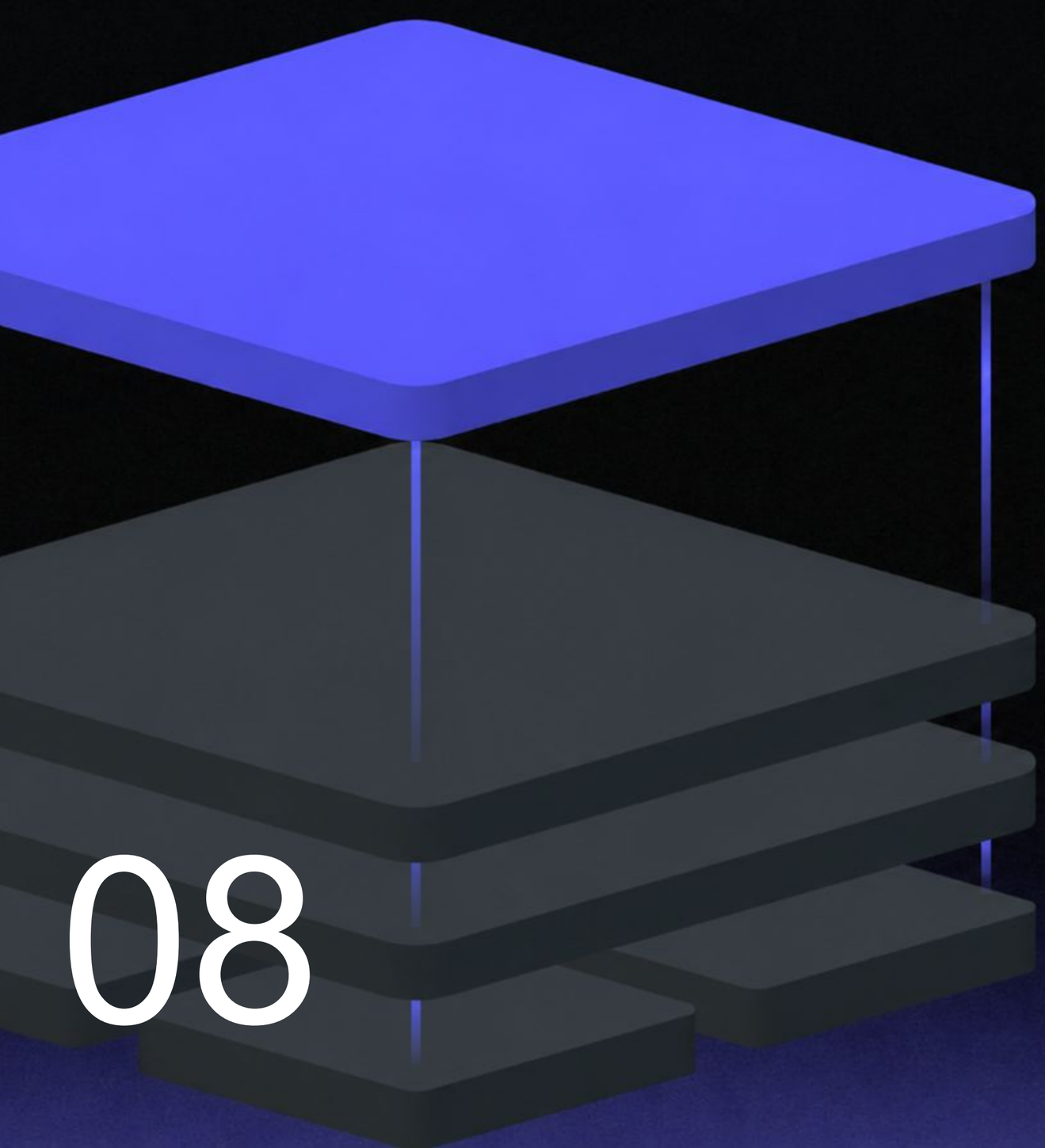
A slice of the prefill rides along with the decode steps — decode keeps its cadence.



prefill chunk · decode · prefill chunk · decode …

Prefill is compute-bound and decode is bandwidth-bound, so mixing them in one batch uses both resources rather than saturating one and idling the other.

The chunk size is a knob against your SLA. Bigger chunks finish prefill sooner; smaller chunks protect time-per-output-token for everyone already streaming.



08

Modular

Modular Cloud, or BYOC

Serving & Modeling APIs

Mojo: The systems language for the AI era

NVIDIA AMD Q

aws intel arm

# Why Modular Cloud?

Modular Cloud manages clusters of accelerators to do LLM serving at scale.

### THE PROBLEM

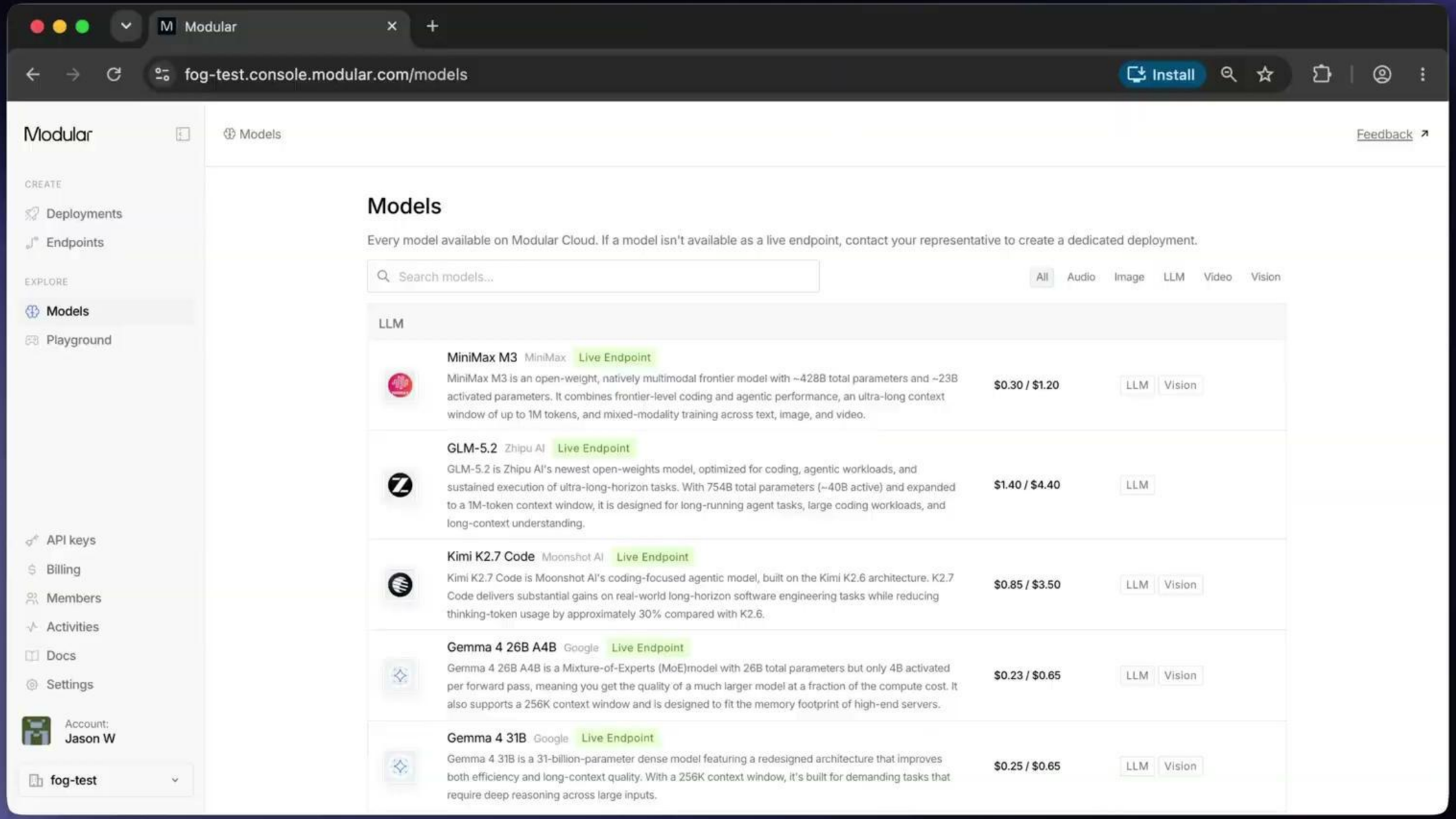
Replicas hold different KV state, prefill and decode want different hardware, and every workload has a different SLA. Round-robin routing throws computed work away.

### THE COMPONENT

A control plane for the fleet, and Mammoth — the Kubernetes-native orchestrator inside each cluster: cache-aware routing, disaggregated prefill and decode, autoscaling.

### THE CONSEQUENCE

Managed endpoints, or the same stack inside your own cloud — across NVIDIA, AMD and Qualcomm clusters at once.


Modular
fog-test.console.modular.com/models
Install
Modular
Models
Feedback
CREATE
Deployments
Endpoints
EXPLORE
Models
Playground
API keys
Billing
Members
Activities
Docs
Settings
Account:
Jason W
fog-test
Models
Every model available on Modular Cloud. If a model isn't available as a live endpoint, contact your representative to create a dedicated deployment.
Search models...
All
Audio
Image
LLM
Video
Vision
LLM
MiniMax M3 MiniMax Live Endpoint
MiniMax M3 is an open-weight, natively multimodal frontier model with ~428B total parameters and ~23B activated parameters. It combines frontier-level coding and agentic performance, an ultra-long context window of up to 1M tokens, and mixed-modality training across text, image, and video.
$0.30 / $1.20
LLM
Vision
GLM-5.2 Zhipu AI Live Endpoint
GLM-5.2 is Zhipu AI's newest open-weights model, optimized for coding, agentic workloads, and sustained execution of ultra-long-horizon tasks. With 754B total parameters (~40B active) and expanded to a 1M-token context window, it is designed for long-running agent tasks, large coding workloads, and long-context understanding.
$1.40 / $4.40
LLM
Kimi K2.7 Code Moonshot AI Live Endpoint
Kimi K2.7 Code is Moonshot AI's coding-focused agentic model, built on the Kimi K2.6 architecture. K2.7 Code delivers substantial gains on real-world long-horizon software engineering tasks while reducing thinking-token usage by approximately 30% compared with K2.6.
$0.85 / $3.50
LLM
Vision
Gemma 4 26B A4B Google Live Endpoint
Gemma 4 26B A4B is a Mixture-of-Experts (MoE)model with 26B total parameters but only 4B activated per forward pass, meaning you get the quality of a much larger model at a fraction of the compute cost. It also supports a 256K context window and is designed to fit the memory footprint of high-end servers.
$0.23 / $0.65
LLM
Vision
Gemma 4 31B Google Live Endpoint
Gemma 4 31B is a 31-billion-parameter dense model featuring a redesigned architecture that improves both efficiency and long-context quality. With a 256K context window, it's built for demanding tasks that require deep reasoning across large inputs.
$0.25 / $0.65
LLM
Vision

# Modular Cloud is a GenAI inference service

Customers integrate an OpenAI-compatible client API and send requests to a model endpoint.

### Enterprise app

Client API · OpenAI spec

→

### Modular Cloud

Control plane — manage the deployment fleet

Data plane — routing, request / response handling

→

### MAX

Accelerator fleet · KV cache · model cache

```
https://api.modular.com/inference/v1
```

# Modular Cloud architecture

Console / API users
deploy and manage

→

Control plane
Dashboard · deployment and endpoint management · billing and usage metering

←

Inference clients
OpenAI API

**NVIDIA GPU cluster · B200**

- mcloud-agent
- Gateway
- Orchestrator
- MAX container

**Qualcomm cluster · AI200**

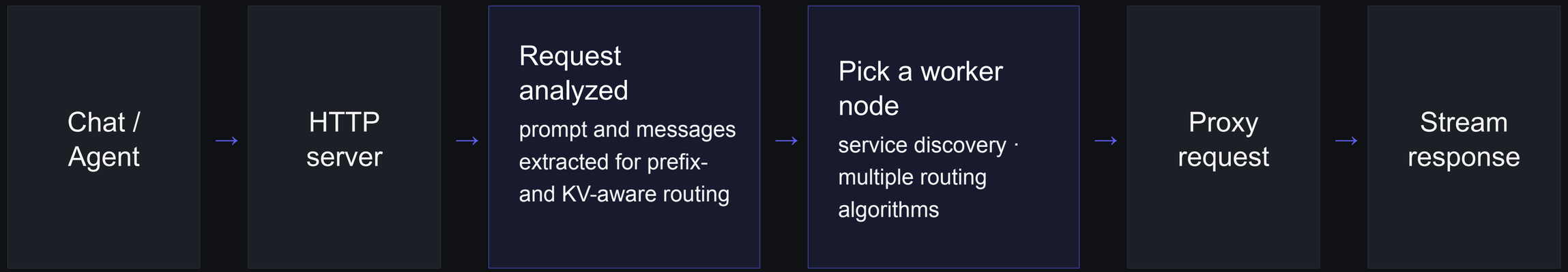
- mcloud-agent
- Gateway
- Orchestrator
- MAX container

**AMD GPU cluster · MI355**

- mcloud-agent
- Gateway
- Orchestrator
- MAX container

# Mammoth: the orchestrator

Every request flows through Mammoth, the orchestrator inside each cluster.



# Picking the worker node

### Requests are wildly unequal

100 tokens or 100k tokens; milliseconds or minutes of generation.

### Workers carry state

Prefill leaves KV cache behind. Reuse only happens if the next turn lands on the worker that holds it.

### Prefill and decode behaviors

Prefill burns compute; decode burns memory bandwidth. The same worker can be full on one and idle on the other.

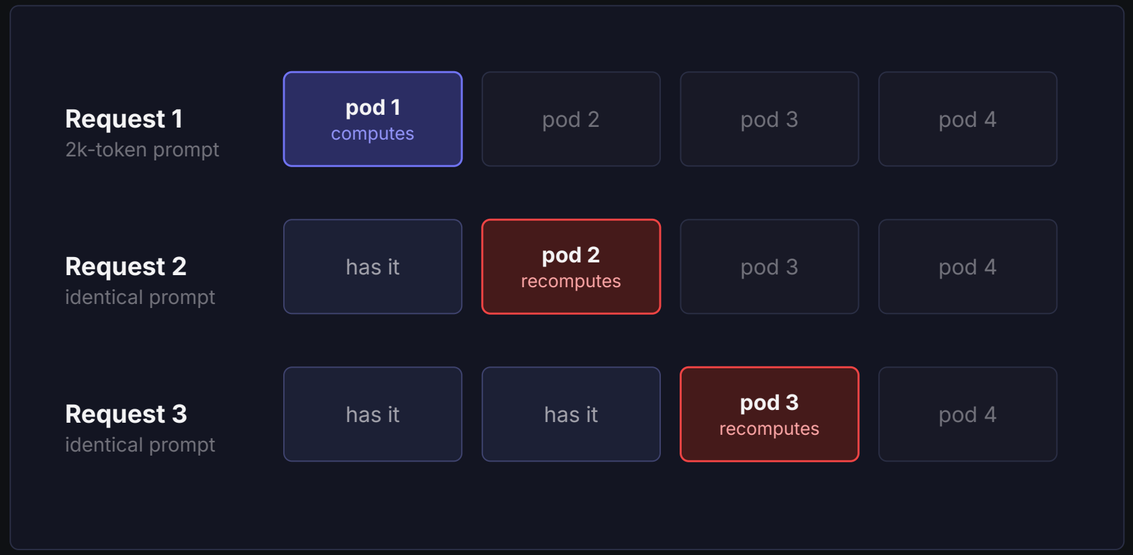
### SLAs differ per workload

Completions want instant TTFT; agents want cheap long contexts; batch jobs want raw throughput. Identical treatment serves nobody.

# Mammoth: routing algorithms

| | | |
|---|---|---|
| 01 | **Round robin** | Routes to each node, one by one. |
| 02 | **Least requests** | Routes to the worker with the fewest in-flight requests. |
| 03 | **Prefix aware** | Uses the request’s prefix to find a matching worker. |
| 04 | **KV aware** | Routes to the worker with the best KV-cache match. |

# Round robin works for stateless services. GenAI inference is not stateless.




## Naive routing reduces cache locality

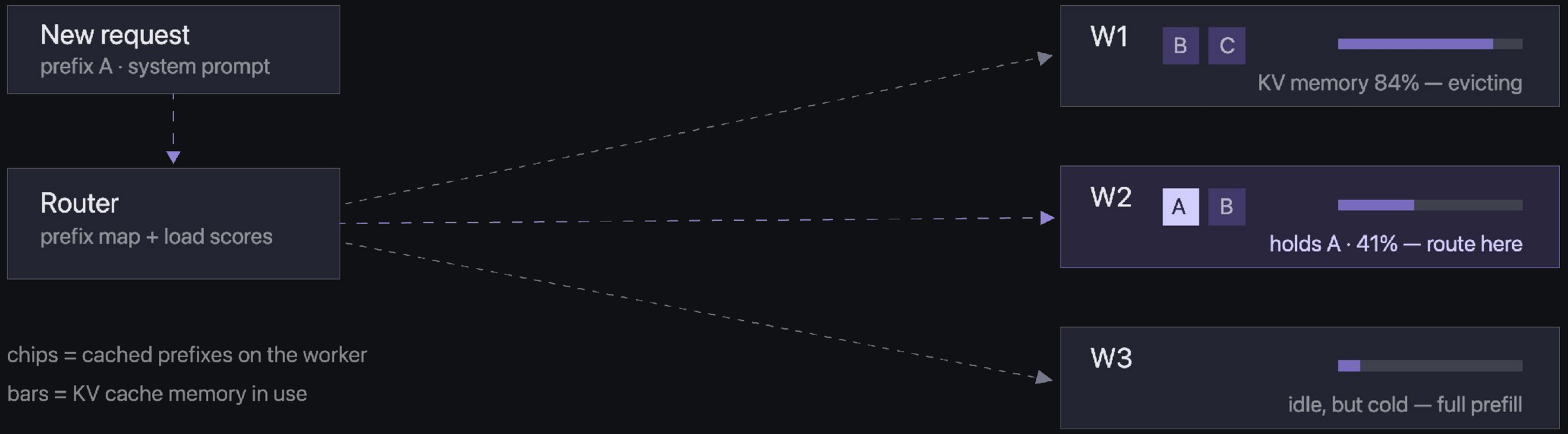
More replicas means a lower chance of landing on the one holding your prefix. Scale-out increases redundant prefill overhead quadratically.

## Cache-aware routing scales perfectly

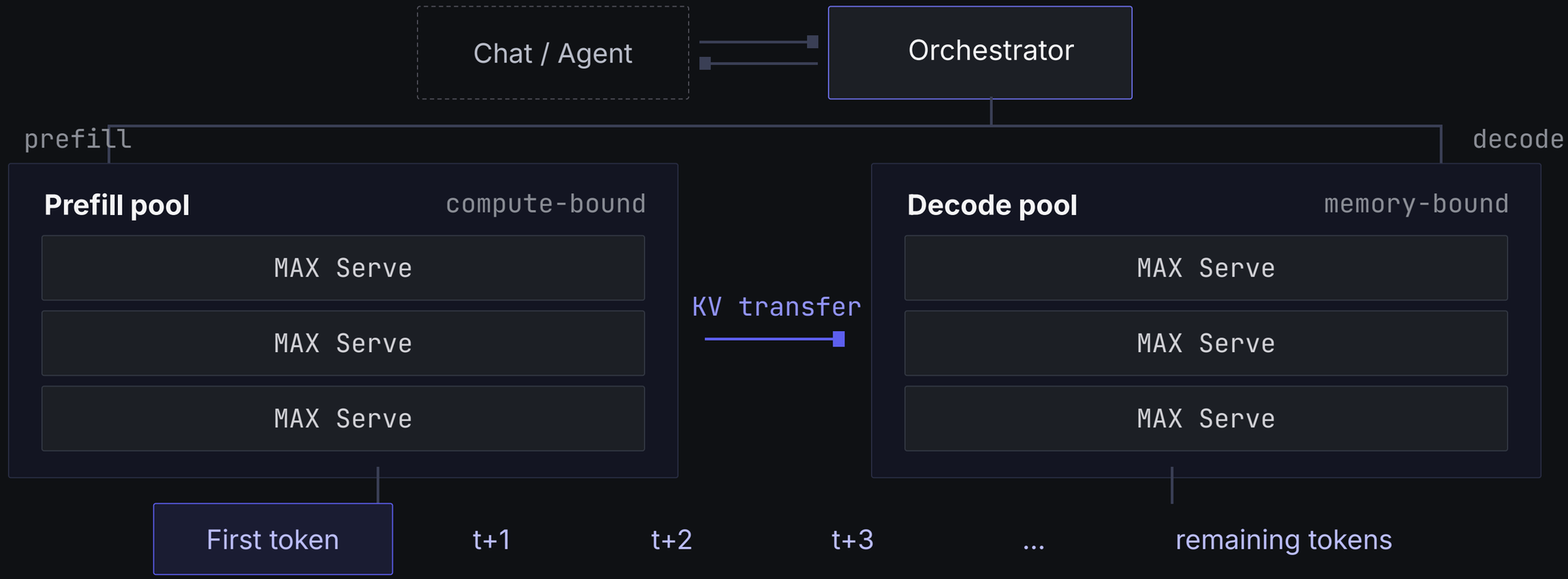
More replicas = more aggregate cache capacity. Larger clusters directly translate to a higher cumulative cache hit rate, reversing the scale-out penalty.

Each replica holds a different KV state. Routing to the wrong one throws away computed work and reruns prefill from scratch.

# KV-cache-aware routing



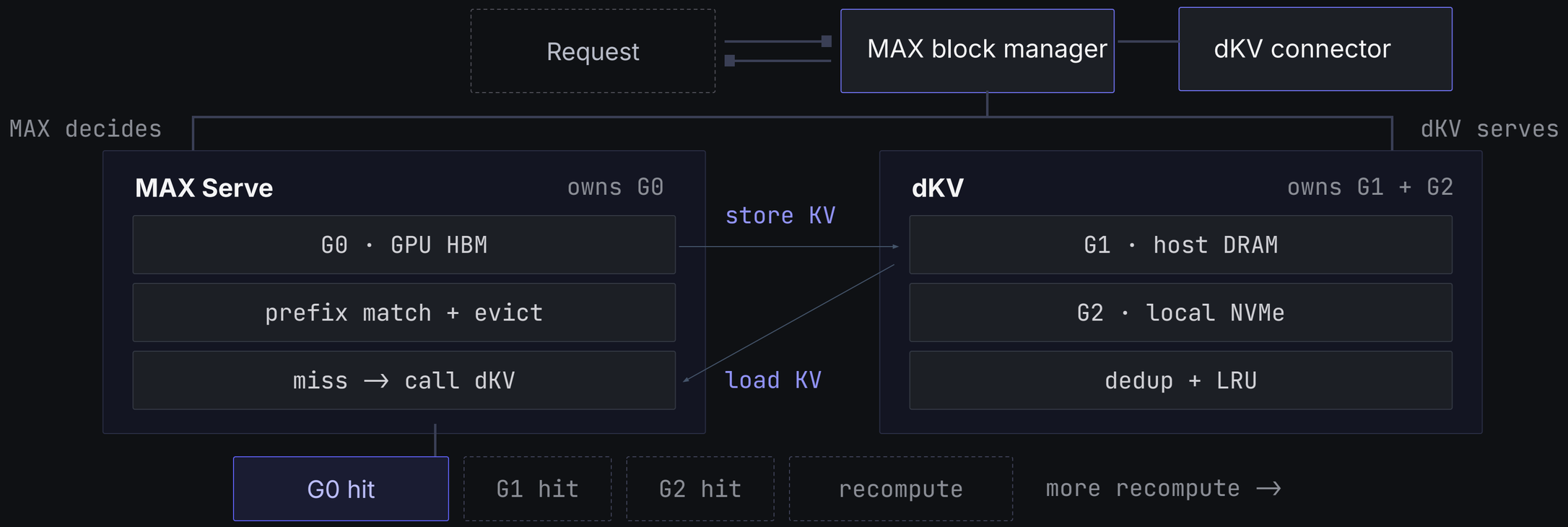
# Mammoth: disaggregated prefill



# KV-cache offloading across the memory hierarchy

| TIER | WHERE | BANDWIDTH | CAPACITY |
|---|---|---|---|
| G0 | GPU HBM | ~TB/s | hundreds of GB |
| G1 | Host DRAM | tens of GB/s | 1–2 TB per node |
| G2 | Local NVMe | single-digit GB/s | tens of TB |
| Peers | Other nodes | RDMA-class | cluster capacity |

# dKV: cluster-wide KV caching across HBM, DRAM and NVMe




peer-to-peer kv-transfers

# 09

# Modular, the token factory

# To solve AI you need to build a unifying platform

| LAYER | YOUR STACK TODAY | THE MODULAR PLATFORM |
|---|---|---|
| **Platform** | No unified cloud layer — DIY deployment | **Modular Cloud** — managed endpoints |
| **Serving** | vLLM · SGLang · TensorRT-LLM | **MAX Serve** — OpenAI-compatible endpoint, scheduler |
| **Modeling and compilation** | PyTorch + torch.compile | **MAX modeling API + graph compiler** — model definitions, fusion, placement |
| **Kernels** | CUDA · CUTLASS · Triton · vendor C++ | **MAX kernel library, in Mojo** — open source, one implementation |
| **Language** | CUDA · CuTe DSL · Triton · HIP · NKI · C++ | **Mojo** — the systems language for the AI era, one language across accelerators |
| **Hardware** | One vendor’s silicon | Any accelerator — NVIDIA, AMD, Apple, Trainium, TPU, Qualcomm |

Modular

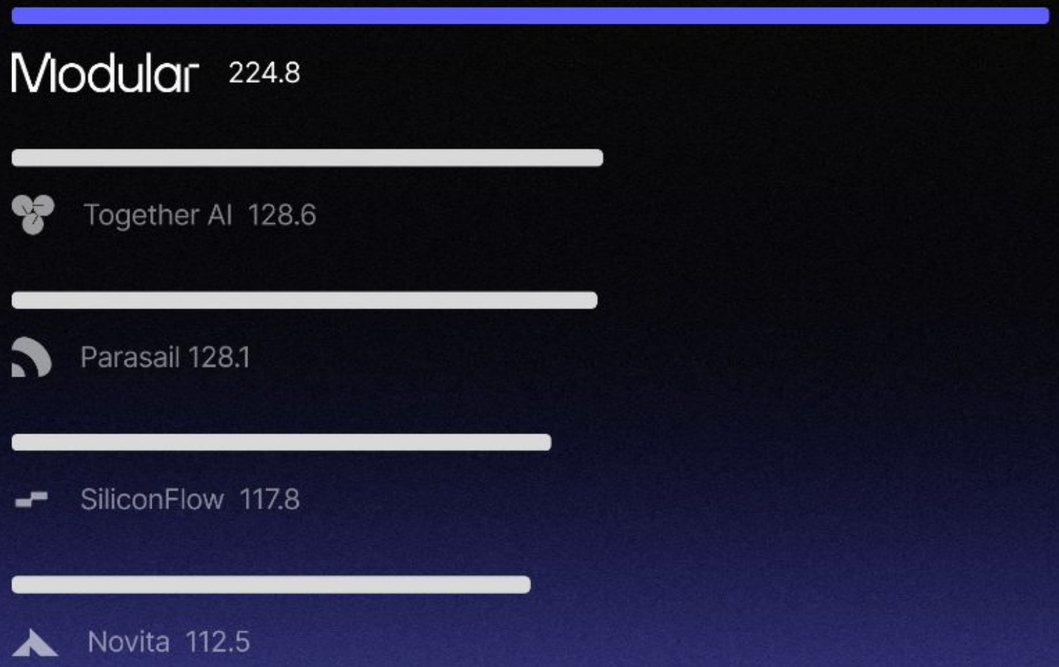
MiniMax M3

Artificial Analysis

# 1st

Fastest

Tokens per second





Flux 2

# 3x

Faster token generation



# Best performance on real production workloads

Modular

MiniMax M3

openrouter

P75 End-to-End Latency

| Provider | Min (s) | Max (s) |
|---|---|---|
| ModelRun [by Modular] | 6.89s | 8.79s |
| MiniMax | 9.31s | 12.51s |
| Together | 8.42s | 15.73s |
| Venice | 9.59s | 14.86s |
| NovitaAI | 10.98s | 16.83s |
| Parasail | 8.27s | 22.09s |
| StreamLake | 12.55s | 18.29s |

Modular

# Both throughput and latency

MiniMax M3

openrouter

P75 Throughout

| Provider | Min (tok/s) | Max (tok/s) | Avg (tok/s) |
|---|---|---|---|
| ModelRun [by Modular] | 154 | 166 | 162 |
| Venice | 127 | 152 | 142 |
| CoreWeace | 79 | 169 | 122 |
| Parasail | 71 | 140 | 99 |
| StreamLake | 83 | 114 | 96 |
| MiniMax | 82 | 98 | 91 |
| GMICloud | 70 | 107 | 88 |

# 10

# Let’s run the token

# One request, all the way down

01 **Tokenize and hash the prompt**
native, model-specific, byte-exact

02 **Ask who holds the prefix**
binary search over a chained-hash index — microseconds

03 **Prepare, filter, score, pick**
twice, if prefill and decode are separate pools — plus the cache hint

04 **Admit into a step**
continuous batching; prefill chunked and mixed with decodes

05 **Stream the cache back in**
from CPU RAM, disk or another node — on a copy stream

06 **Run the graph**
one MEF artifact, buffers already planned, launches already paid for

07 **Run the fused kernels**
paged flash attention, MegaFFN, norms and quantisation inside epilogues

08 **Execute instructions**
the tile schedule the compiler folded flat, on whichever chip this is

09 **Verify the speculated tokens**
keep the agreed prefix, resample at the first disagreement

10 **Stream out, register the blocks**
the index learns where this conversation now lives, for the next turn

# Thank you & Questions

**Read the code**

The compiler, the standard library and every kernel in this talk are open source under Apache 2.0.

**Join us for the next deep dives**

Scheduled over the next few months and will cover each of these components in a lot more depth.

**SCHEDULE AND RECORDINGS**

`go/ModularTechStack`

| TECH TALK | DATE |
|---|---|
| Mojo compiler and stdlib | Sep 30, 2026 |
| Kernels | Oct 7, 2026 |
| Graph compiler, runtime and device context | Oct 14, 2026 |
| MAX framework | Oct 21, 2026 |
| MAX Serve | Oct 28, 2026 |
| MAX and Mojo tooling | Nov 4, 2026 |
| Distributed inference | Nov 18, 2026 |
| Performance engineering | Dec 2, 2026 |